Randomized Algorithms (RA-MIRI)
Assignment 3: Cardest

Roger Canal Estrada, roger.canal@estudiantat.upc.edu

Decembre 2025
Course 2025/2026

Contents

Chapter 1

Introduction

In this assignment, we were asked to do an experimental study of some Cardinality Estimation Algorithms. The idea was to study various of those, but instead I focused more on only the two suggested and doing more detailed experiments.

In this report, I implement, analyze, and compare these two algorithms:

1. **HyperLogLog (HLL):** A register-based algorithm relying on the observation of leading zeros in binary hash representations.
2. **Recordinality (REC):** An order-statistic algorithm relying on the k-th smallest values (records) in a random permutation.

The implementation was done in Python and this is the link to the GitHub repository with all the scripts used and the results shown [**link**]

Chapter 2

HyperLogLog

2.1 Algorithm Overview

HyperLogLog, presented in the assignment by Flajolet et al. in 2007 [1], is an algorithm for cardinality estimation. The core intuition is that in a uniform stream of random integers, a pattern of ρ leading zeros occurs with probability $2^{-\rho}$. Therefore, observing a maximum of ρ zeros suggests a cardinality of roughly 2^{ρ}.

2.2 Stochastic Averaging

Relying on a single observation of leading zeros is highly volatile; a single "lucky" hash with 30 zeros would imply a cardinality of billions (a billion-to-one shot).

To mitigate this, HLL employs **Stochastic Averaging**. The input stream is split into m independent substreams (buckets) using the first b bits of the hash.

$$m = 2^b \quad \text{with} \quad b \in [4 \dots 16]$$

The algorithm maintains m registers, each tracking the maximum leading zeros seen in its bucket. Averaging these registers reduces the standard error of the estimate. According to the Central Limit Theorem, the standard error decreases proportional to $1/\sqrt{m}$.

2.3 The Harmonic Mean & Error Formula

A key innovation of HLL is the use of the **Harmonic Mean** instead of the Geometric or Arithmetic mean to combine the register values.

- The Arithmetic Mean is susceptible to outliers. One bucket with a "lucky" large value would drastically inflate the total estimate.
- The Harmonic Mean suppresses the effect of these outliers.

Flajolet derived the standard error of HLL as:

$$\text{Standard Error} \approx \frac{1.04}{\sqrt{m}} \tag{2.1}$$

The constant 1.04 is derived from the complex integral of the Harmonic Mean's variance distribution (specifically involving $\log_2$).

2.4 Alpha Correction

The raw Harmonic Mean is a biased estimator (it tends to underestimate). To correct this, a multiplicative factor α_m is applied. As defined in [1]:

$$E = \alpha_m \cdot m^2 \cdot \left(\sum_{j=1}^{m} 2^{-M[j]} \right)^{-1} \tag{2.2}$$

Where α_m is determined by the number of registers m:

$$\alpha_m = \begin{cases} 0.673 & \text{for } m = 16 \\ 0.697 & \text{for } m = 32 \\ 0.709 & \text{for } m = 64 \\ \frac{0.7213}{1+1.079/m} & \text{for } m \geq 128 \end{cases}$$

Chapter 3

Recordinality

3.1 Algorithm Overview

Recordinality, presented in the assignment by Helmi et al. [2], approaches the problem via queueing the hash values of the stream as a random permutation.

The algorithm maintains a buffer of size k containing the k-largest (or smallest) hash values seen so far. The core metric is R, the number of times the set of records has "evolved" (i.e., a new element was inserted, displacing an old one).

3.2 Estimation Formula

The relationship between the number of updates R and the cardinality n is exponential. The estimator is given by:

$$\hat{n} = k \cdot \left(1 + \frac{1}{k}\right)^{R-k} \tag{3.1}$$

3.3 Memory Implications

Unlike HLL, which stores small integers (5 bits) in registers, Recordinality must store the *actual hash values* to maintain the sorted order and check for duplicates. If I use 32-bit hashes, each unit of memory cost is significantly higher than HLL.

Chapter 4

Experiments

4.1 Experimental Setup

To ensure a scientific comparison, I tested both algorithms under **Equal Memory Constraints**.

- **HLL Cost:** $m \times 5$ bits (assuming max register value < 32).
- **REC Cost:** $k \times 32$ bits (storing full integer hashes).

Table 4.1: Parameter Alignment for Equal Memory

HLL Bits (b)	**HLL Registers** (m)	**Approx Memory (bits)**	**REC Size** (k)
4	16	80	2
6	64	320	10
10	1024	5120	160
14	16384	81920	2560

All experiments were conducted using a custom Python implementation. To guarantee robustness, every data point represents the **mean of 10 independent trials** using different random seeds for the hash family. The hash functions were generated using the `randomhash` library [3].

4.2 Real Data Experiments

To evaluate performance on real-world distributions, I tested the algorithms on two real datasets (given by the assignment with a prepared structure).

- **Dataset A:** *Dracula*. Ground Truth Cardinality: $n \approx 9,425$.
- **Dataset B:** *Crusoe*. Ground Truth Cardinality: $n \approx 6,245$.

4.2.1 Results Analysis

The results, averaged over 10 independent trials, are summarized in Tables 4.2 and 4.3.

Consistency Across Datasets: The performance is striking in its consistency. In both datasets, HyperLogLog (HLL) significantly outperforms Recordinality (REC) at low memory tiers.

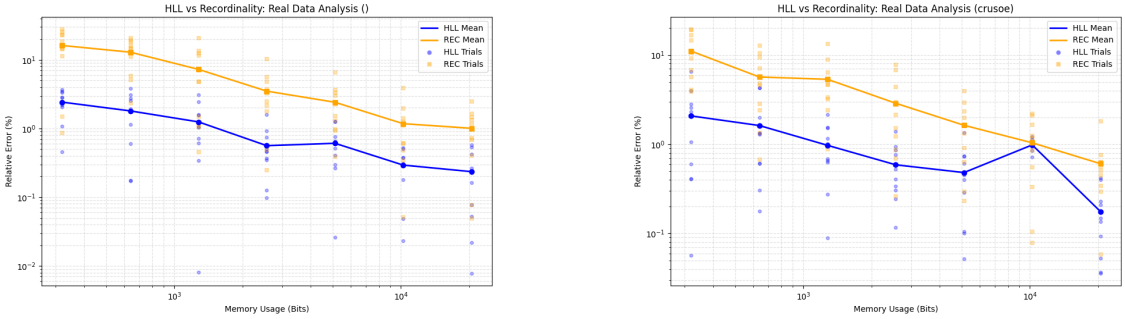
- **Low Memory ($b = 6, \approx 320$ bits):** HLL maintains an error of $\approx 2\%$, while Recordinality collapses to an error rate of $10\% - 11\%$. This confirms that Recordinality struggles when k is small ($k = 10$), as the sample size is insufficient to stabilize the estimator.
- **Standard Memory ($b = 10, \approx 5$ kb):** HLL achieves high precision ($< 0.5\%$ error). Recordinality improves significantly here (dropping to $\approx 1.7\%$ error) but remains roughly $3\times$ less accurate than HLL for the same memory cost.

Table 4.2: Results for *Dracula* (True $n = 9,425$)

Bits (b)	Memory	REC (k)	HLL Error %	REC Error %	REC Std Dev %
6	320	10	2.12%	9.87%	7.41%
8	1280	40	0.58%	6.27%	5.35%
10	5120	160	**0.34%**	1.80%	1.83%
12	20480	640	0.27%	0.70%	0.49%

Table 4.3: Results for *Crusoe* (True $n = 6,245$)

Bits (b)	Memory	REC (k)	HLL Error %	REC Error %	REC Std Dev %
6	320	10	2.08%	11.08%	5.79%
8	1280	40	0.97%	5.34%	3.41%
10	5120	160	**0.48%**	1.63%	1.15%
12	20480	640	0.18%	0.61%	0.45%



(a) Dracula Results (b) Crusoe Results

Figure 4.1: Comparative Accuracy on Real Datasets (10 trials).

HLL (Blue) shows lower error than REC (Orange), particularly at lower memory settings.

4.3 Synthetic Data Experiments

To rigorously test the algorithms, I generated synthetic data using a Zipfian distribution generator. This allows control over stream length (N), cardinality (n), and skew (α).

4.3.1 Experiment A: Scale Impact (N and n)

I tested the robustness of the algorithms against the scale of the data using the $b = 10$ configuration.

Impact of Stream Length (N)

Fixing $n = 5,000$ and increasing N to $1,000,000$:

- **HLL (Stability):** Extremely stable with error remaining in the $0.3\% - 0.7\%$ range. The algorithm is structurally insensitive to duplicates; once the unique elements have updated the registers, subsequent duplicate arrivals do not alter the internal state.
- **REC (Volatility):** Shows noticeable fluctuations ($1.2\% - 2.5\%$). While theoretically insensitive to duplicates, the higher variance of Recordinality means that across different random seeds (trials), the estimates disperse more widely.

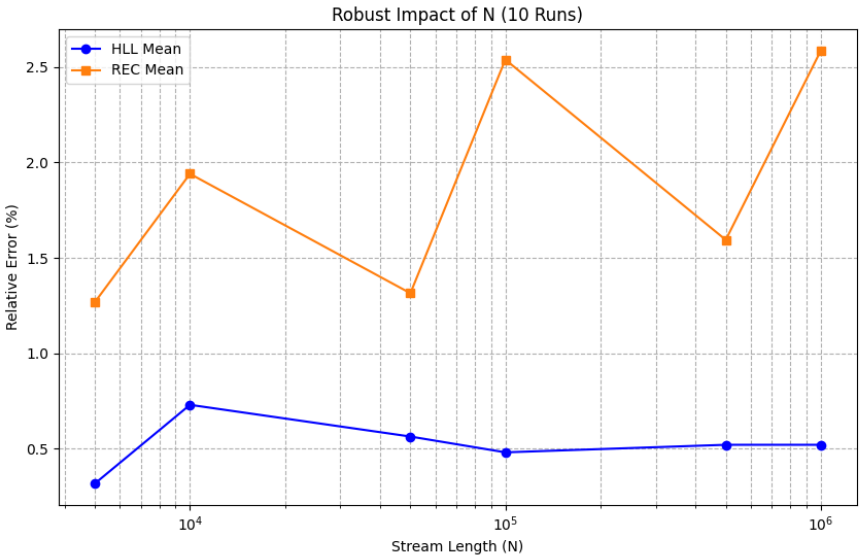


Figure 4.2: Impact of Stream Length (N) on Error. HLL remains stable (flat), while REC shows higher variability.

Analysis of Behavior: The stability gap is a direct consequence of the sample density. Under equal memory constraints ($b = 10$), HyperLogLog aggregates information from $m = 1024$ independent registers, whereas Recordinality relies on a sample of only $k = 160$ records.

Impact of Cardinality (n)

Fixing memory ($b = 10, k = 160$) and varying n:

- **Deterministic Phase ($n \leq k$):** When the cardinality is small, Recordinality functions as a hybrid data structure. Since the buffer of size $k = 160$ is not yet full, it stores every unique element explicitly. Consequently, the estimator is **exact** (0.00% error).
- **Probabilistic Phase ($n > k$):** Once the distinct count exceeds the buffer size ($n > 160$), the algorithm transitions to probabilistic estimation. Old records are displaced by new ones, and the error jumps to the expected range of $\approx 1.5\% - 3\%$.

Table 4.4: Impact of Cardinality n on Error ($b = 10, k = 160$)

True Cardinality	HLL Error %	REC Error %
100	0.61%	**0.00% (Exact)**
1,000	0.40%	1.44%
10,000	0.36%	3.33%
80,794	0.47%	2.90%

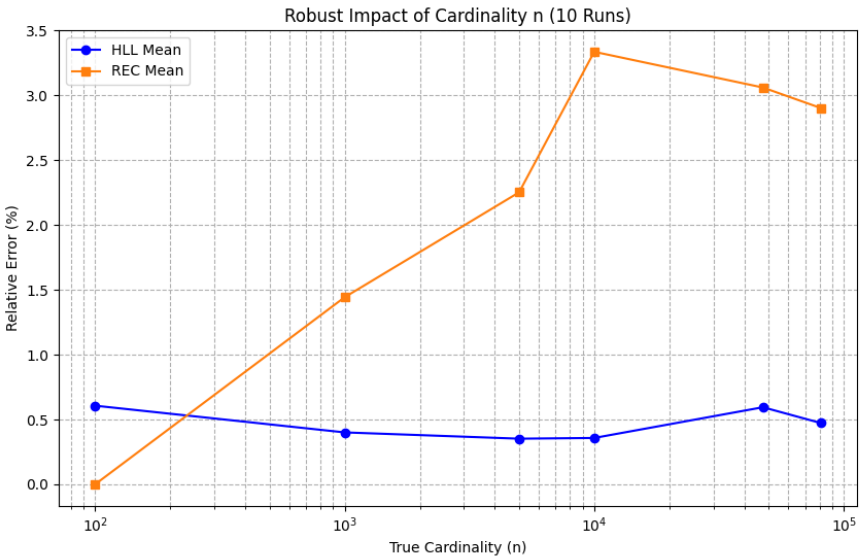


Figure 4.3: Impact of Cardinality (n). Note the transition from exact counting to estimation when $n > 160$.

Analysis of Behavior: The drop to 0% error is not an anomaly but a structural feature of Recordinality. Unlike HyperLogLog, which is always probabilistic regardless of n, Recordinality acts as a simplified Hash Set until its memory limit is reached. The implementation explicitly handles this bifurcation:

```
# Algorithm State Check:
# If the buffer is not full, we are in the Deterministic Phase.
# The count R is exactly equal to the number of unique elements seen.
if r < self.k:
    estimates.append(float(r))
```

```
    # If buffer is full, we switch to the Probabilistic Phase.
    # We apply the exponential expansion formula.
    else:
        exponent = r - self.k
        est = self.k * (base ** exponent)
        estimates.append(int(est))
```

This "small n" efficiency makes Recordinality potentially superior for applications where the cardinality is frequently small but occasionally explodes to large numbers.

4.3.2 Experiment B: Accuracy vs Memory on Synthetic Data

Mirroring the methodology used for the real datasets, this experiment isolates the algorithms' performance by removing the irregularities of natural language. I used the controlled Zipfian stream with fixed parameters to test the impact of memory allocation (b) exclusively.

Fixed Parameters:

- Stream Length: $N = 100,000$
- True Cardinality: $n = 10,000$
- Skew: $\alpha = 1.0$

I varied the parameter b from 4 to 14, creating a memory range from 80 bits to over 80,000 bits.

Results Analysis

The results, summarized in Figure 4.4 and Table 4.5, highlight a critical divergence in stability between the two algorithms.

- **HyperLogLog (Stability):** The error decreases smoothly following the theoretical $1/\sqrt{m}$ curve. Even at very low memory ($b = 4, \approx 80$ bits), HLL maintains a usable estimate with $\approx 3\%$ error.
- **Recordinality (Low-Memory Explosion):** Recordinality exhibits extreme instability at low memory. At $b = 4$ ($k = 2$), the standard deviation is nearly 20%. This confirms that Recordinality requires a minimum "critical mass" of memory ($k > 10$) to stabilize the exponential nature of its estimator $\hat{n} \approx k(1 + 1/k)^{R-k}$.

Table 4.5: Synthetic Accuracy vs Memory ($n = 10,000$)

b	Memory (bits)	REC k	HLL Error %	REC Error %	REC Std Dev %
4	80	2	3.22%	38.85%	20.18%
6	320	10	1.72%	14.90%	7.71%
10	5120	160	**0.32%**	1.85%	0.88%
14	81920	2560	0.08%	0.23%	0.19%

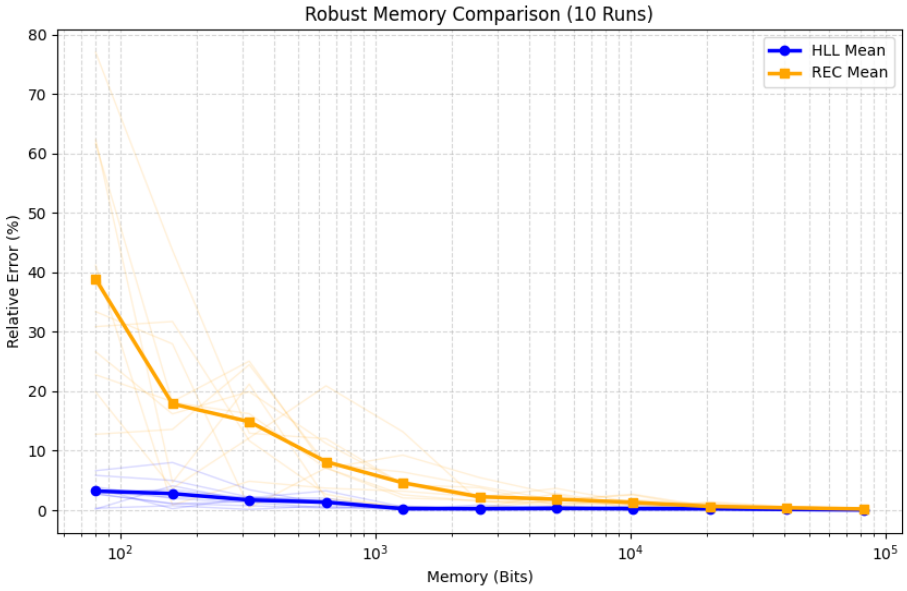


Figure 4.4: Error vs Memory Usage (Log Scale) on Synthetic Data.

HLL (Blue) shows strictly tighter variance than REC (Orange), particularly in the low-memory region (left side).

Chapter 5

Conclusion

This comparative analysis demonstrates that **HyperLogLog is the superior algorithm** for general-purpose cardinality estimation in large-scale data streams, while **Recordinality** exhibits specific utility in low-cardinality regimes.

5.1 Summary of Findings

5.1.1 Memory Efficiency and Accuracy

HyperLogLog consistently outperforms Recordinality when normalized for memory usage.

- **High Density Information:** By storing only the "maximum leading zeros" (requiring ≈ 5 bits per register), HLL packs significantly more estimators into the same memory budget than Recordinality, which must store full 32-bit hash values.
- **Quantifiable Gain:** At a standard memory budget of 5 KB ($b = 10$), HyperLogLog achieves an error rate of $\approx 0.3\% - 0.6\%$, whereas Recordinality lags at $\approx 1.8\% - 2.4\%$ with the same memory footprint.

5.1.2 Stability and Variance

A critical finding of this study is the **structural instability** of Recordinality in probabilistic mode.

- **HyperLogLog:** Exhibits a stable, low-variance error profile across all experiments (N, n, and α). The harmonic mean of $m = 1024$ registers effectively suppresses statistical outliers.
- **Recordinality:** Suffers from high variance, particularly at low memory ($k < 20$). The exponential nature of its estimator ($\hat{n} \approx k(1 + 1/k)^{R-k}$) means that small fluctuations in the record count R translate into massive jumps in the estimated cardinality.

Bibliography

[1] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm," *DMTCS Proceedings*, 2007.

[2] A. Helmi, J. Lumbroso, C. Martínez, and A. Viola, "Data Streams as Random Permutations: the Distinct Element Problem," *DMTCS Proceedings*, 2012.

[3] J. Lumbroso. (2025). *python-random-hash: A Python library for random hash families*. GitHub Repository. Available: https://github.com/jlumbroso/python-random-hash